VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL - ELECTRONIC ENGINEERING
DEPARTMENT OF ELECTRONICS

# Thesis Proposal

## Design of a Scalable Hybrid I/O CGRA with Multicast Routing for Efficient Edge SNN Inference

**Instructor:**

Assoc. Prof., PhD. Truong Quang Vinh

**Students:**

Huynh Pham Anh Duy – 2051095
Trieu Thanh Vinh – 2213980

Ho Chi Minh City, January 2026

## Abstract

This thesis presents the architectural definition and implementation of a Coarse-Grained Reconfigurable Architecture (CGRA) optimized for Spiking Neural Network (SNN) inference. Modern event-driven workloads suffer from irregular data access and significant I/O bottlenecks. To address these challenges, the proposed accelerator features a novel "Hybrid I/O" architecture that decouples high-bandwidth input streaming (DMA) from low-latency control and status output (APB), effectively minimizing bus contention and reducing synchronization overhead.

A key innovation of the design is the integration of a parameterizable mesh of Processing Elements (PEs) interconnected by a dynamic multicast-capable routing fabric. This fabric exploits the high fan-out nature of SNNs by replicating spike packets to multiple destination PEs in a single cycle, significantly reducing latency compared to traditional point-to-point networks. Each PE includes a hardware-accelerated Leaky Integrate-and-Fire (LIF) unit to compute neuron dynamics with high energy efficiency.

The system is implemented on the Xilinx Zynq-7000 XC7Z020 SoC using a hardware-software co-verification strategy. Key technical contributions include a double-buffered configuration memory for zero-latency reconfiguration and a deterministic scheduling mechanism for sparse dataflow execution. Experimental results confirm a post-synthesis maximum operating frequency of 213.72 MHz (on Xilinx UltraScale+ reference target) with functional deployment verified at 50 MHz on the Zynq-7000 XC7Z020, and a 100% pass rate across 8,915 regression tests (25 suites), establishing a scalable and robust platform for edge-based neuromorphic computing.

# Contents

**Abstract** **i**

**List of Abbreviations** **v**

**1 Introduction** **1**
1.1 Overview of CGRA and SNN Inference . . . . . 1
1.2 System Context and Execution Flow . . . . . 1
1.3 Target Workload and Problem Scope . . . . . 2

**2 Literature Review and Background** **3**
2.1 Overview of Coarse-Grained Reconfigurable Architectures . . . . . 3
2.2 Spiking Neural Networks (SNNs) . . . . . 3
2.2.1 Leaky Integrate-and-Fire (LIF) Neuron Model . . . . . 3
2.2.2 Information Encoding Schemes . . . . . 3
2.3 Computational Advantages of CGRA for SNNs . . . . . 4

**3 Project Objectives** **5**
3.1 Design Goals . . . . . 5
3.2 Evaluation Criteria . . . . . 5

**4 Proposed System Architecture** **6**
4.1 SoC System Integration: The Hybrid I/O Architecture . . . . . 6
4.2 Accelerator Subsystems . . . . . 7
4.2.1 Global Control and CSR . . . . . 7
4.2.2 Double-Buffered Configuration Memory . . . . . 7
4.2.3 Bi-Directional AXI4 DMA Engine . . . . . 7
4.2.4 Row-Banked Tile Memory . . . . . 7
4.3 Processing Element Tile Design . . . . . 8
4.4 Parametric Design Generation and Scalability . . . . . 9
4.5 Instruction Set Architecture (ISA) . . . . . 9
4.6 Configuration Frame Format . . . . . 9
4.7 Opcode Summary . . . . . 10
4.8 Hardware Signal Interface and Port Definitions . . . . . 10
4.9 Top-Level Module (cgra_top.sv) . . . . . 12
4.10 DMA Engine (cgra_dma_engine.sv) . . . . . 13
4.11 Processing Element (cgra_pe.sv) . . . . . 15
4.12 Tile Memory (cgra_tile_memory.sv) . . . . . 16

**5 Verification and Testing** **19**
5.1 Verification Methodology: Industry-Standard Rigor . . . . . 19
5.2 Verification Flow . . . . . 19
5.3 Regression Test Suites . . . . . 20
5.4 Verification Infrastructure Hardening . . . . . 20
5.5 Hardware-Software Co-Verification . . . . . 21
5.6 Results and Coverage Closure . . . . . 21

**6 Implementation Results** **22**
6.1 Synthesis and Resource Utilization . . . . . 22
6.2 Timing Analysis . . . . . 22
6.3 Power Consumption . . . . . 23

**7 Conclusion and Future Directions** **24**
7.1 Summary of Contributions . . . . . 24
7.2 Future Directions . . . . . 24

**8 Tentative Thesis Outline** **25**

**9 Project Timetable** **26**

**References** **27**

## List of Figures


1 Biological vs. Artificial Spiking Neuron Models . . . . . 1
2 Hybrid I/O Architecture: Decoupled Control (APB) and Data (AXI4) Planes 6
3 High-Level Block Diagram of the CGRA Accelerator IP Subsystems . . . . 7
4 Bi-Directional AXI4 DMA Engine Architecture . . . . . 8
5 4x4 PE Array Mesh Topology . . . . . 9
6 Multicast NoC Routing Patterns in the 4x4 Mesh . . . . . 10
7 Top-Level RTL Module Hierarchy . . . . . 12
8 DMA Engine Control Logic and AXI4 Interface . . . . . 14
9 PE Datapath and Instruction Execution Pipeline . . . . . 15
10 Row-Banked Tile Memory Architecture . . . . . 17


## List of Tables


1 Configuration Frame Field Descriptions . . . . . 11
2 ALU Opcode Summary: Arithmetic and Logical Operations . . . . . 11
3 cgra_top.sv Signal Definitions . . . . . 13
4 DMA Engine Signal Definitions . . . . . 14
5 Processing Element Signal Definitions . . . . . 16
6 Tile Memory Signal Definitions . . . . . 17
7 Hardware Resource Utilization (Reference Synthesis: Zynq UltraScale+ XCZU4EV) . . . . . 22
8 Detailed Project Timeline . . . . . 26

## List of Abbreviations

| | |
|---|---|
| **ALU** | Arithmetic Logic Unit |
| **AMBA** | Advanced Microcontroller Bus Architecture |
| **ANN** | Artificial Neural Network |
| **APB** | Advanced Peripheral Bus |
| **ASIC** | Application-Specific Integrated Circuit |
| **AXI** | Advanced eXtensible Interface |
| **BPTT** | Backpropagation Through Time |
| **CGRA** | Coarse-Grained Reconfigurable Architecture |
| **CPU** | Central Processing Unit |
| **CSC** | Compressed Sparse Column |
| **CSR** | Control and Status Register |
| **DDR** | Double Data Rate (Memory) |
| **DMA** | Direct Memory Access |
| **DRAM** | Dynamic Random Access Memory |
| **DVS** | Dynamic Vision Sensor |
| **FIFO** | First-In, First-Out |
| **FPGA** | Field-Programmable Gate Array |
| **FSM** | Finite State Machine |
| **GPU** | Graphics Processing Unit |
| **IP** | Intellectual Property |
| **LIF** | Leaky Integrate-and-Fire |
| **LUT** | Lookup Table |
| **MAC** | Multiply-Accumulate |
| **NoC** | Network on Chip |

| | |
|---|---|
| **PE** | Processing Element |
| **RAM** | Random Access Memory |
| **RF** | Register File |
| **SIMD** | Single Instruction, Multiple Data |
| **SNN** | Spiking Neural Network |
| **SoC** | System on Chip |
| **SPM** | Scratchpad Memory |
| **SRAM** | Static Random Access Memory |
| **STDP** | Spike-Timing-Dependent Plasticity |
| **VLIW** | Very Long Instruction Word |

# 1 Introduction

## 1.1 Overview of CGRA and SNN Inference

Spiking Neural Networks (SNNs) represent the third generation of neural computation [1], providing energy-efficient execution through event-driven mechanisms. However, deploying SNNs on traditional hardware (CPUs/GPUs) is inefficient due to sparse, irregular connectivity. SNNs operate on discrete spike events, causing SIMD utilization to drop sharply in von Neumann processors.

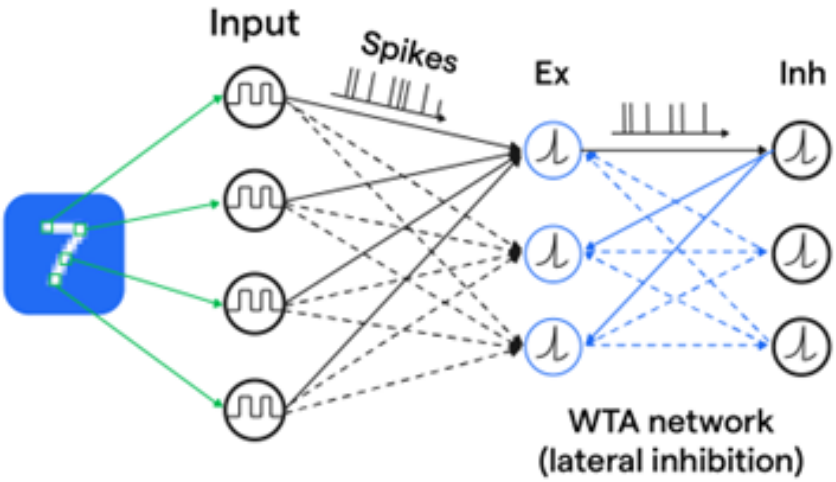


Figure 1: Biological vs. Artificial Spiking Neuron Models

To address these limitations, the proposed CGRA employs a spatially programmable 4×4 mesh of Processing Elements (PEs). Each tile combines a fixed-point Multiply-Accumulate (MAC) unit with a hardwareimplemented Leaky IntegrateandFire (LIF) neuron update engine. This unified execution model allows the array to compute sparse matrix–vector (matvec) operations and neuron dynamics in a tightly coupled, dataflow-driven manner.

## 1.2 System Context and Execution Flow

The CGRA is designed as a tightly integrated hardware accelerator under full control of the dual-core ARM Cortex-A9 processing system. The typical execution flow consists of:

1. **Configuration Stage:** The host processor writes bitstream (context words) into the CGRA configuration memory via the CSR interface. These bitstream encode the spatial and temporal behavior of the 4×4 PE mesh.

2. **Data Preparation:** SNN weight matrices are stored in Compressed Sparse Column (CSC) format in DDR memory. Input spike vectors and neuron states are prepared and aligned for DMA-based transfer.

3. **Execution Phase:** The CGRA performs sparse matvec operations and LIF neuron updates in each PE according to the dynamical equation: $v[t+1] = \beta \cdot v[t] + I[t] - \lambda$.

4. **Result Collection:** Updated membrane potentials and spike outputs are written back to DDR for use by the next simulation timestep or software post-processing.

### 1.3 Target Workload and Problem Scope

The CGRA accelerator is optimized for sparse SNN layer updates, specifically weight matrices $W \in \mathbb{R}^{128\times128}$ stored in CSC format. This representation allows the accelerator to process only active presynaptic spikes by scanning the corresponding nonzero column entries, thus avoiding dense computation. The primary bottlenecks addressed are:

- **Routing Congestion:** fan-out dominant distributions causing "Routing Black Holes."
- **Data Starvation:** irregular memory access patterns leading to PE idle cycles.
- **Precision Inefficiency:** inefficient use of bit-width in standard generic ALUs, requiring specialized 40-bit saturating logic for high-fidelity LIF updates.

The system architecture and DMA engine are designed according to the AMBA AXI Standard [2], ensuring interoperability with the Zynq-7000 processing system [3].

# 2 Literature Review and Background

## 2.1 Overview of Coarse-Grained Reconfigurable Architectures

Coarse-Grained Reconfigurable Architectures (CGRAs) represent a class of programmable hardware structures positioned between traditional fine-grained FPGAs and fixed-function ASICs. Well-known examples include ADRES [4] and the more recent Plasticine architecture [5]. Extensive surveys on CGRA performance and architectural diversity can be found in [6] and [7].

CGRAs mitigate issue of routing congestion and configuration overhead by operating at the arithmetic-operator granularity. A typical CGRA execution flow involves configuration mapping, spatial/temporal scheduling, and data routing across the mesh.

## 2.2 Spiking Neural Networks (SNNs)

SNNs represent the third generation of neural network models [1], in which information is encoded and processed using discrete spike events rather than continuous-valued activations. This temporal and event-driven formulation, as detailed in [8], allows SNNs to more closely resemble biological neural systems. Modern deep learning applications leveraging these models are extensively discussed in [9] and [10], while training methodologies such as surrogate gradient learning [11] have bridged the gap between SNNs and traditional deep learning.

### 2.2.1 Leaky Integrate-and-Fire (LIF) Neuron Model

Among many neuron models, the LIF model strikes a balance between biological plausibility and implementation simplicity. It captures the essential features of integration, leakage, and threshold-based spiking. In discrete time, the LIF neuron is modeled as:

$$v[t+1] = \beta \cdot v[t] + I[t] - \lambda \tag{1}$$

where $v[t]$ is the membrane potential, $\beta \in (0, 1)$ is the leak factor, $I[t]$ is the sum of weighted spikes, and $\lambda$ is a constant modeling tonic leak.

### 2.2.2 Information Encoding Schemes

There are two primary encoding paradigms supported by this architecture:

- **Rate Coding:** Information is conveyed by the firing rate of the neurons.
- **Temporal Coding:** Information is encoded in the precise timing of individual spikes (Time-to-First-Spike).

### 2.3 Computational Advantages of CGRA for SNNs

The theoretical motivation for designing a specific CGRA for SNNs lies in:

- **Event-Driven Processing:** Computation is only triggered when an input spike arrives, saving dynamic power.
- **Sparsity Exploitation:** By using CSC format and multicast routing, the architecture avoids fetching "zero" values.
- **Locality of Reference:** Keeping the membrane potential state $v[t]$ in local PE scratchpads reduces data movement energy.

The SNN implementation and dataset verification also leverage standard benchmarks like MNIST [12] and modern software frameworks such as snnTorch [13].

# 3 Project Objectives

The primary objective of this project is to design and implement a CGRA-based accelerator IP featuring an APB control interface and an AXI4 high-bandwidth DMA engine.

## 3.1 Design Goals

- **High-Dynamic-Range Computation:** Development of a 4×4 PE mesh incorporating 40-bit saturating arithmetic.
- **Neuromorphic Acceleration:** Integration of dedicated hardware LIF units within each PE for efficient potential evolution and spike generation.
- **Low-Latency Context Switching:** Implementation of a "hot-reloadable" configuration mechanism using double-buffered context memory.
- **Scalable Sparse Processing:** Support for Compressed Sparse Column (CSC) data structures to optimize for connectivity sparsity (<1% activity).

## 3.2 Evaluation Criteria

The success of the project will be measured by:

1. Correct functional behavior as verified by an 8,915-case regression suite (25 suites, directed + constrained-random).
2. Protocol compliance with AMBA AXI4 and APB standards as validated by assertions.
3. Timing closure at 50 MHz on the Xilinx Zynq-7000 XC7Z020 (AXI4/APB) with post-synthesis $F_{max}$ analysis demonstrating >200 MHz feasibility.
4. Demonstration of end-to-end inference for digit classification tasks (MNIST).

# 4 Proposed System Architecture

The system architecture is partitioned into four major subsystems designed to maximize data throughput and minimize stall cycles.

## 4.1 SoC System Integration: The Hybrid I/O Architecture

The CGRA IP is integrated into the Programmable Logic of the Xilinx Zynq-7000 XC7Z020 SoC. To overcome the common "bottleneck black hole" found in standard accelerator designs, this project proposes a **Hybrid I/O architecture** that decouples data and control paths:

- **AXI4 Data Plane (DMA):** Facilitates high-bandwidth, burst-oriented communication with DDR3 DRAM for weight and spike streaming.
- **APB Control Plane (CSR):** A dedicated low-latency bus for register-mapped configuration, status monitoring, and interrupt handling, ensuring control transactions never stall the data pipeline.

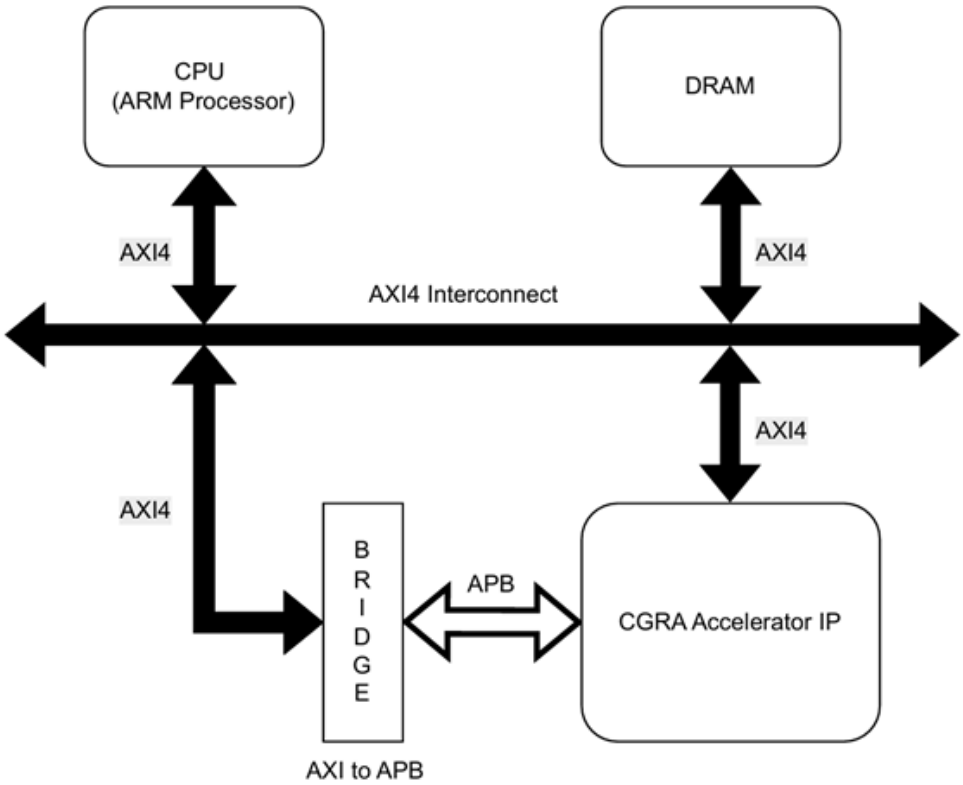


Figure 2: Hybrid I/O Architecture: Decoupled Control (APB) and Data (AXI4) Planes

## 4.2 Accelerator Subsystems

### 4.2.1 Global Control and CSR

This module decodes commands from the host via the APB slave interface. It manages interrupt generation, status monitoring, and the accelerator's lifecycle.

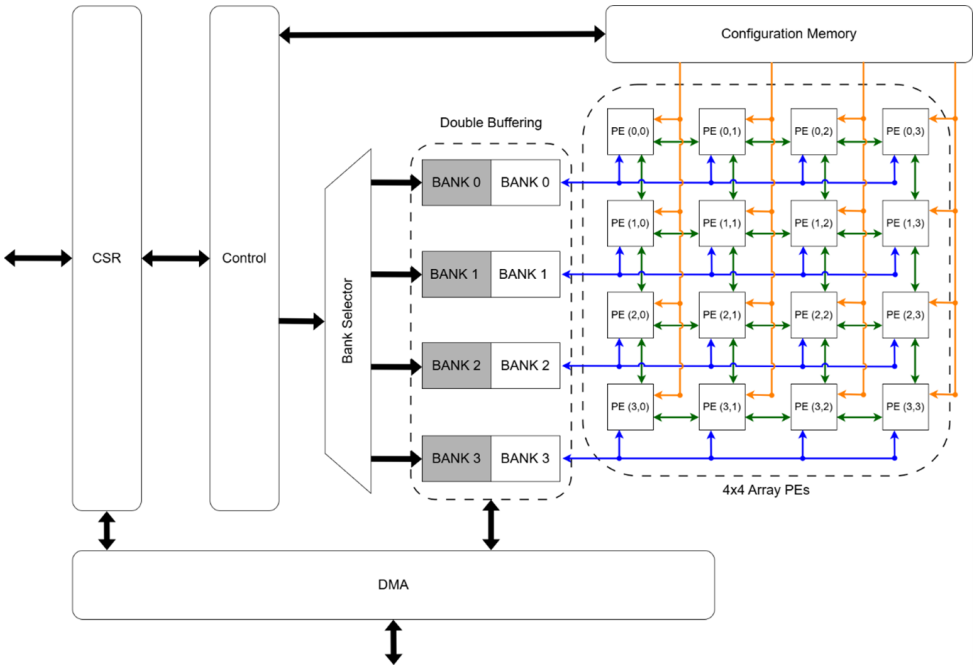


Figure 3: High-Level Block Diagram of the CGRA Accelerator IP Subsystems

### 4.2.2 Double-Buffered Configuration Memory

To support rapid reconfiguration required by multi-layer networks, the configuration memory employs a double-buffering scheme. This allows the DMA to pre-fetch the bitstream for the next layer (the "back" buffer) while the current layer is executing from the "front" buffer.

### 4.2.3 Bi-Directional AXI4 DMA Engine

The DMA unit operates as an AXI4 Master with **backpressure-aware flow control**. Unlike traditional input-only designs, this engine features a dedicated Write Engine to offload intermediate neuron states back to DDR. This enables the execution of multi-layer SNNs that exceed on-chip memory capacity without CPU-side data copying.

### 4.2.4 Row-Banked Tile Memory

The global tile memory is physically partitioned into four independent SRAM banks. Each bank is dedicated to a specific row of the 4×4 PE array, allowing all four rows to fetch operands simultaneously without contention.

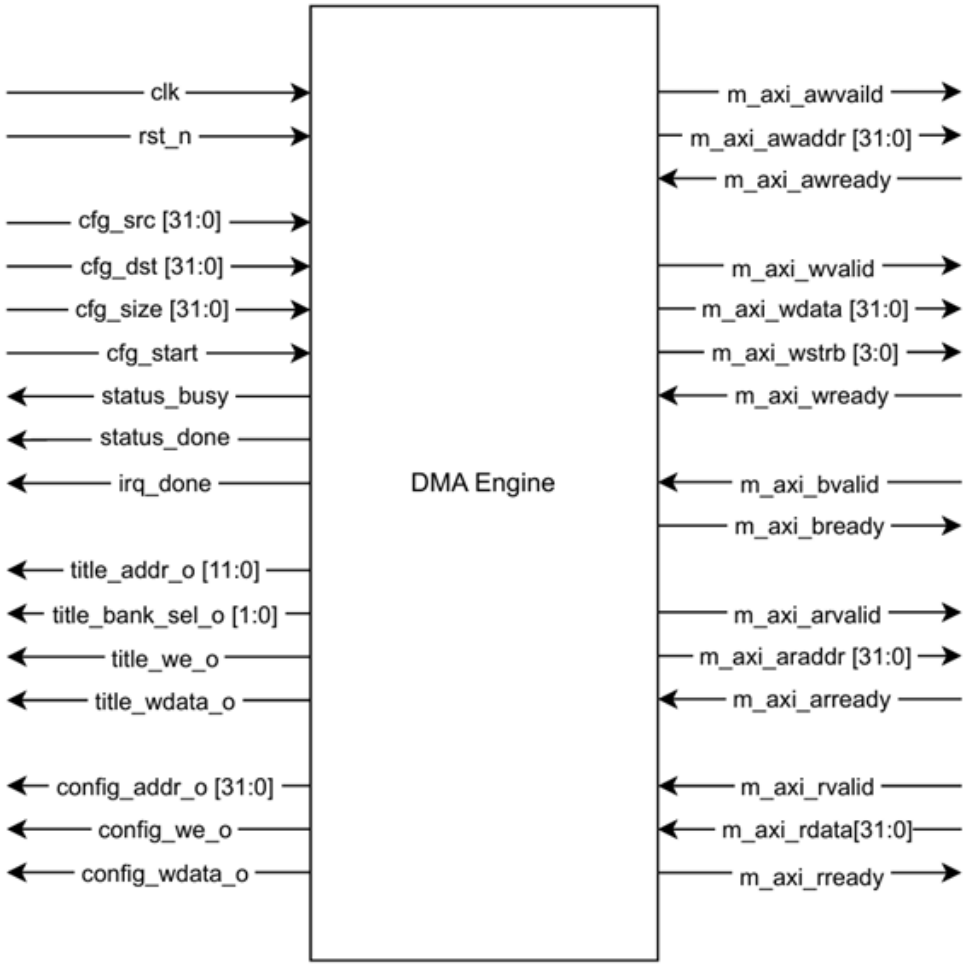


Figure 4: Bi-Directional AXI4 DMA Engine Architecture

### 4.3 Processing Element Tile Design

Each PE tile integrates a compute datapath, local scratchpad memory, and a **Multicast-Enabled Network-on-Chip (NoC)** router. The PE datapath consists of:

- **Decode:** Configuration frame sets opcode and route mask.
- **Operand Select:** Dual 4-to-1 muxes for source operand selection (RF, Neighbors, SPM, Immediate).
- **ALU:** 40-bit saturating MAC unit and 32-bit ALU.
- **LIF Unit:** Discretized neuron update logic.
- **Router (Multicast-Enabled NoC):** A circuit-switched mesh router. It implements a **dynamic bitmask-based multicast algorithm**, allowing a single spike packet to be replicated to any combination of North, East, South, or West neighbors in a single cycle. This is critical for SNNs with high synaptic fan-out.

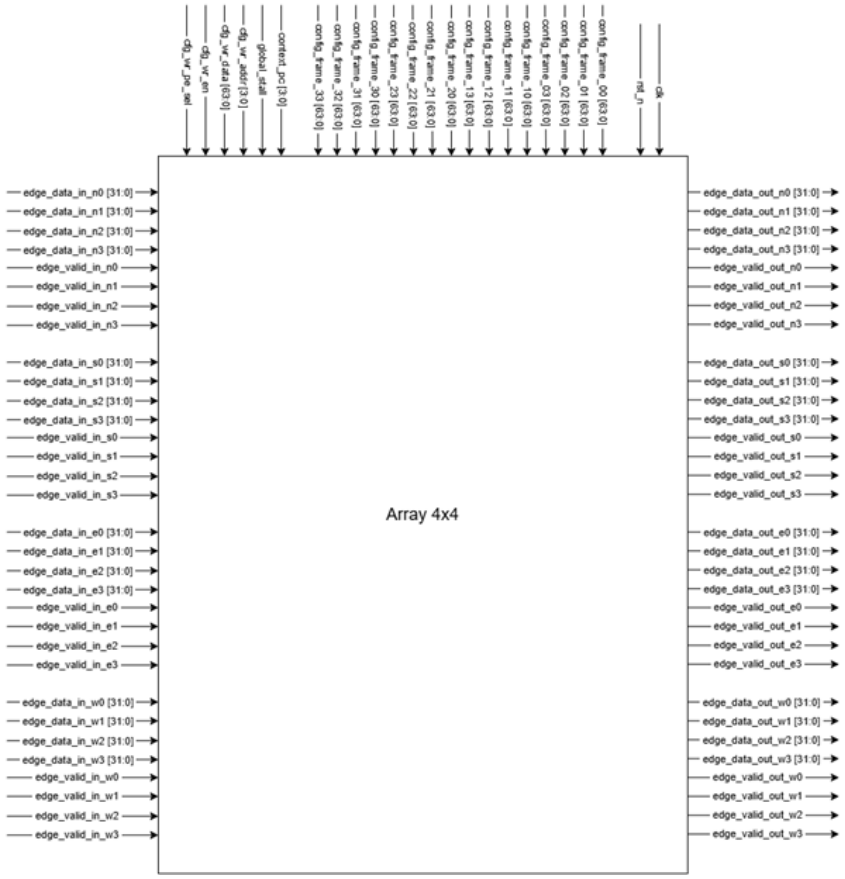


Figure 5: 4x4 PE Array Mesh Topology

### 4.4 Parametric Design Generation and Scalability

To ensure the proposed architecture is not limited to a fixed 4×4 configuration, the project includes the development of a **parametric RTL generation framework**. Built using SystemVerilog parameters and Python-based templating, this framework allows for design space exploration by automatically generating NxM arrays (e.g., 8×8 or 16×16) with correctly sized NoC routing tables and DMA stride logic.

### 4.5 Instruction Set Architecture (ISA)

The CGRA operates on a 64-bit Very Long Instruction Word (VLIW) architecture. Each Processing Element (PE) executes one instruction per clock cycle, defined by a 64-bit configuration frame loaded into its local context memory. The ISA supports 21 distinct operations ranging from standard arithmetic to specialized neuromorphic and ANN activation functions.

### 4.6 Configuration Frame Format

The 64-bit configuration word controls the PE's datapath, including operand selection, ALU operation, destination routing, and immediate value generation. The bit mapping

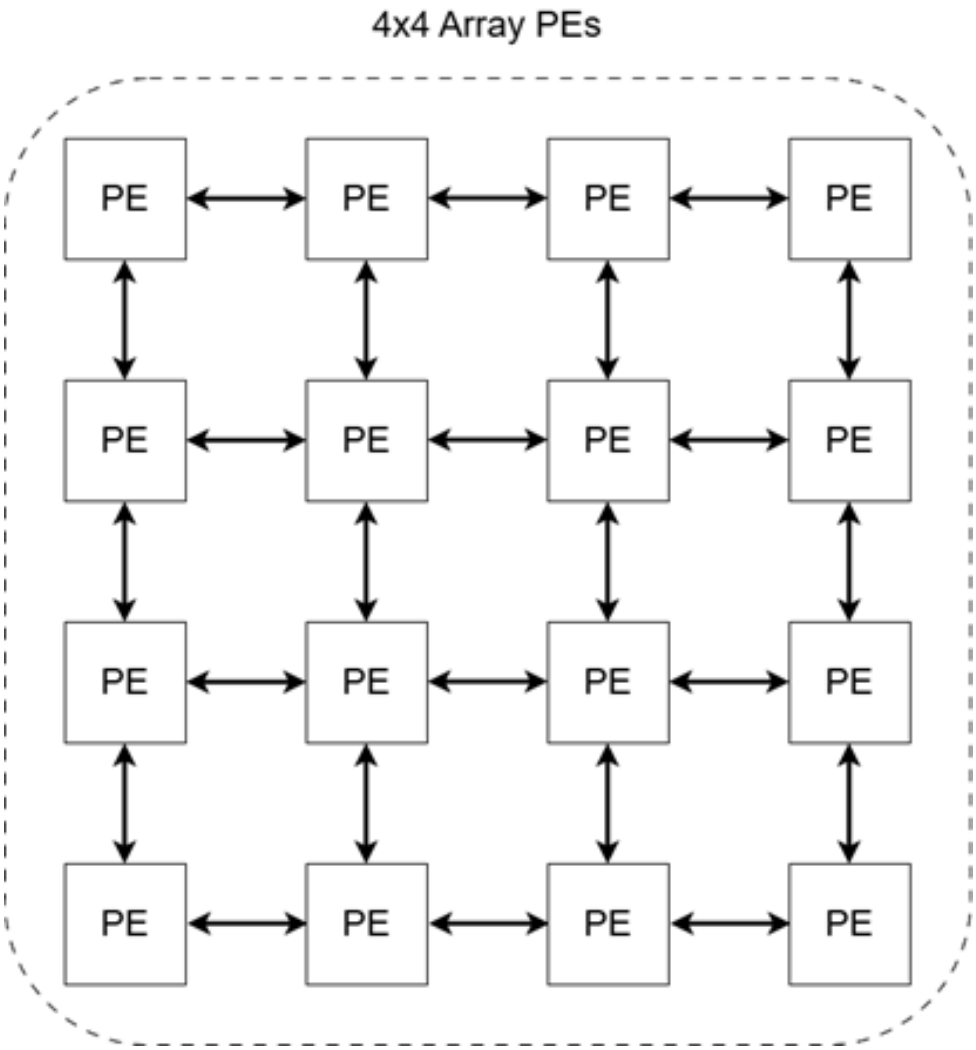


Figure 6: Multicast NoC Routing Patterns in the 4x4 Mesh

is detailed in Table 1.

### 4.7 Opcode Summary

The ALU supports 21 operations categorized into arithmetic, logic, flow control, neuromorphic, and ANN activation. All operations execute in a single cycle.

### 4.8 Hardware Signal Interface and Port Definitions

The hardware implementation of the Scalable Hybrid I/O CGRA adheres to industry-standard communication protocols to ensure seamless integration into modern SoC ecosystems. This section defines the signal-level requirements for the four primary RTL modules: the Top-Level Wrapper, the Bi-Directional DMA Engine, the Processing Element (PE), and the Row-Banked Tile Memory.

To maintain high architectural reliability and timing closure, the design follows three fundamental signaling principles:

Table 1: Configuration Frame Field Descriptions

| **Field** | **Bits** | **Width** | **Description** |
|---|---|---|---|
| opcode | [5:0] | 6 | ALU Operation Code |
| src0_sel | [9:6] | 4 | Source Select for Operand A (RF, Neighbors, SPM, Imm) |
| src1_sel | [13:10] | 4 | Source Select for Operand B |
| dst_sel | [17:14] | 4 | Write-back address for Local Register File (RF) |
| route_mask | [21:18] | 4 | Multicast Output Mask (N, E, S, W) |
| pred_en | [22] | 1 | Enable Predicated Execution |
| pred_inv | [23] | 1 | Invert Predicate Logic |
| imm | [39:24] | 16 | Signed 16-bit Immediate Value |
| cfg_dest_x | [43:40] | 4 | XY routing X coordinate (unicast destination) |
| branch_target | [47:44] | 4 | Dynamic branch PC target (context slot 0–15) |
| branch_en | [48] | 1 | Enable dynamic branch from PE predicate to CU PC |
| data_mode | [50:49] | 2 | SIMD precision: 00=INT32, 01=INT16×2, 10=INT8×4 |
| reserved | [63:51] | 13 | Reserved |

Table 2: ALU Opcode Summary: Arithmetic and Logical Operations

| **Op** | **Mnemonic** | **Description** | **Op** | **Mnemonic** | **Description** |
|---|---|---|---|---|---|
| 0 | NOP | No Operation | 10 | CMP_GT | $(A > B)\ ?\ 1 : 0$ |
| 1 | ADD | A + B (Saturating) | 11 | CMP_LT | $(A < B)\ ?\ 1 : 0$ |
| 2 | SUB | A - B (Saturating) | 12 | CMP_EQ | $(A == B)\ ?\ 1 : 0$ |
| 3 | MUL | $A \times B$ (64-bit) | 13 | LOAD_SPM | $RF \leftarrow SPM[A]$ |
| 4 | MAC | Acc += $A \times B$ (40-bit) | 14 | STORE_SPM | $SPM[A] \leftarrow B$ |
| 5 | AND | A & B (Bitwise AND) | 15 | ACC_CLR | Acc = 0 |
| 6 | OR | A \| B (Bitwise OR) | 16 | PASS0 | Output = A |
| 7 | XOR | $A \oplus B$ (Bitwise XOR) | 17 | PASS1 | Output = B |
| 8 | SHL | $A \ll B[4:0]$ (LSL) | 18 | LIF | Spike = LIF(v, i) |
| 9 | SHR | $A \gg B[4:0]$ (ASR) | 19 | RELU | max(0, A) |
| | | | 20 | MAX | max(A, B) signed |

1. **Protocol Standardization:** All control transactions use the **AMBA APB 2.0** protocol for low-power, non-pipelined configuration access. High-bandwidth data movement is governed by the **AMBA AXI4** (Full) specification, supporting burst transactions and independent read/write channels.

2. **Handshake-Driven Data Flow:** All internal data paths and inter-PE NoC links utilize a **Ready/Valid (Backpressure-Aware)** handshake mechanism. This ensures zero data loss during congestion by stalling upstream producers when downstream consumers are busy.

3. **Unified Synchronous Domain:** All modules operate within a single global clock domain with an active-low synchronous reset (`rst_n`), simplifying the integration into FPGA synthesis flows and preventing Meta-Stability issues across the PE mesh.

The following subsections detail the port definitions for each module, where signal widths marked as "Var" are parameterized to support architectural scaling from 4×4 up to 16×16 arrays.

### 4.9 Top-Level Module (cgra_top.sv)

The top-level module serves as the primary system wrapper, encapsulating the 4×4 PE array, the bi-directional DMA engine, and the APB-based control unit. It manages the global clock and reset distribution while providing the physical interfaces for SoC-level integration.

The wrapper includes a unified address decoder that maps the 32-bit APB address space to the internal Control Status Registers (CSRs), allowing the host CPU to monitor hardware status, configure DMA descriptors, and trigger execution cycles. Additionally, the top-level unit handles interrupt aggregation, combining DMA-completion and Execution-done signals into a single level-triggered IRQ line for the ARM processor.

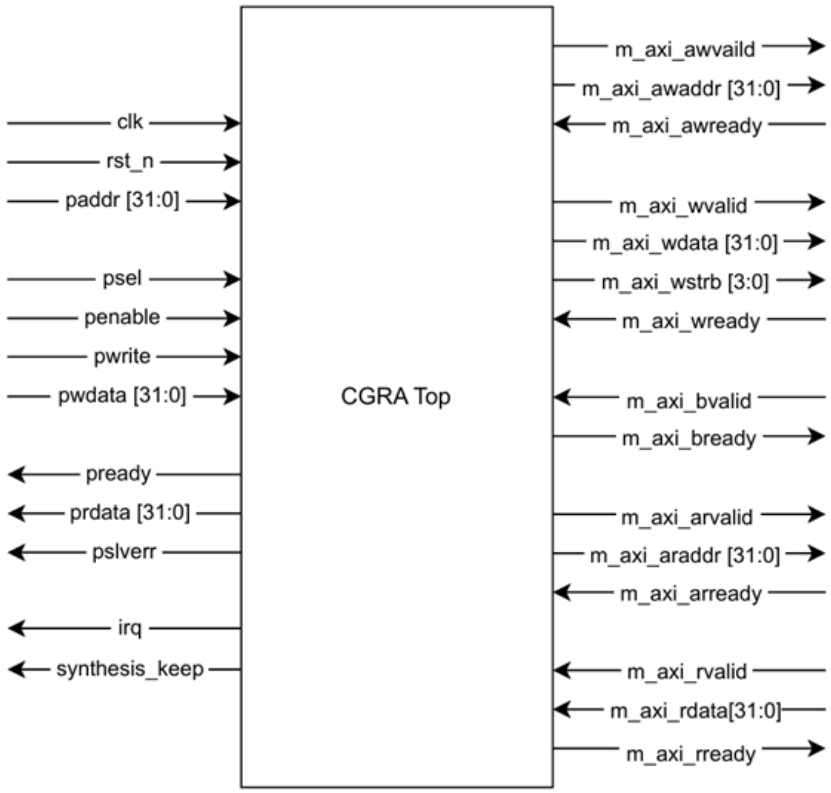


Figure 7: Top-Level RTL Module Hierarchy

Table 3: cgra_top.sv Signal Definitions

| Signal | Dir | Width | Description |
|---|---|---|---|
| clk | I | 1 | System clock |
| rst_n | I | 1 | Active-low synchronous reset |
| psel, penable | I | 1 | APB control signals |
| pwrite | I | 1 | APB write enable |
| paddr | I | Var | APB address bus |
| pwdata, prdata | I/O | 32 | APB data buses |
| m_axi_* | O/I | Var | AXI4 Master interface for DMA |
| irq | O | 1 | Interrupt request |

**Silicon Integration Considerations:** The top-level port list is kept purposely minimal to reduce the complexity of the AXI-to-NoC bridge. The current implementation uses a 32-bit APB address/data bus and a 32-bit AXI4 data bus, matched to the Zynq-7000 HP slave port width. During physical implementation on the Zynq-7000 XC7Z020 fabric, these ports are mapped to the High-Performance (HP) AXI Slave ports of the dual-core ARM Cortex-A9 processing system, enabling cache-coherent memory access. The inclusion of a dedicated interrupt line allows for an event-driven software model, where the CPU remains in a low-power sleep state until the CGRA signals completion of a neural layer update.

### 4.10 DMA Engine (cgra_dma_engine.sv)

The DMA engine is a high-performance, burst-capable AXI4 master responsible for orchestrating data movement between the shared DDR3 memory and the local tile memories. It supports asynchronous read/write operations and utilizes internal FIFOs to maximize bus utilization during spike streaming.

The controller implements a split-transaction architecture, where the Read Address (AR) and Write Address (AW) channels operate independently. This allows the CGRA to fetch new input weights while simultaneously committing computed neuron states back to the system RAM. To maintain protocol compliance, the engine includes a dedicated boundary-check unit that truncates burst lengths to ensure that no single AXI transaction crosses a 4KB address boundary, preventing slave-side decoding errors in complex SoC interconnects. Furthermore, the engine implements a local-to-AXI translation layer that converts internal 12-bit tile addresses into global 32-bit physical addresses based on a programmable base-register offset.

**Design Rationale and Data Integrity:** The decision to use a bi-directional engine over two independent mono-directional units was driven by the need for synchronized memory access. By sharing the AXI master logic, the CGRA can guarantee that weight

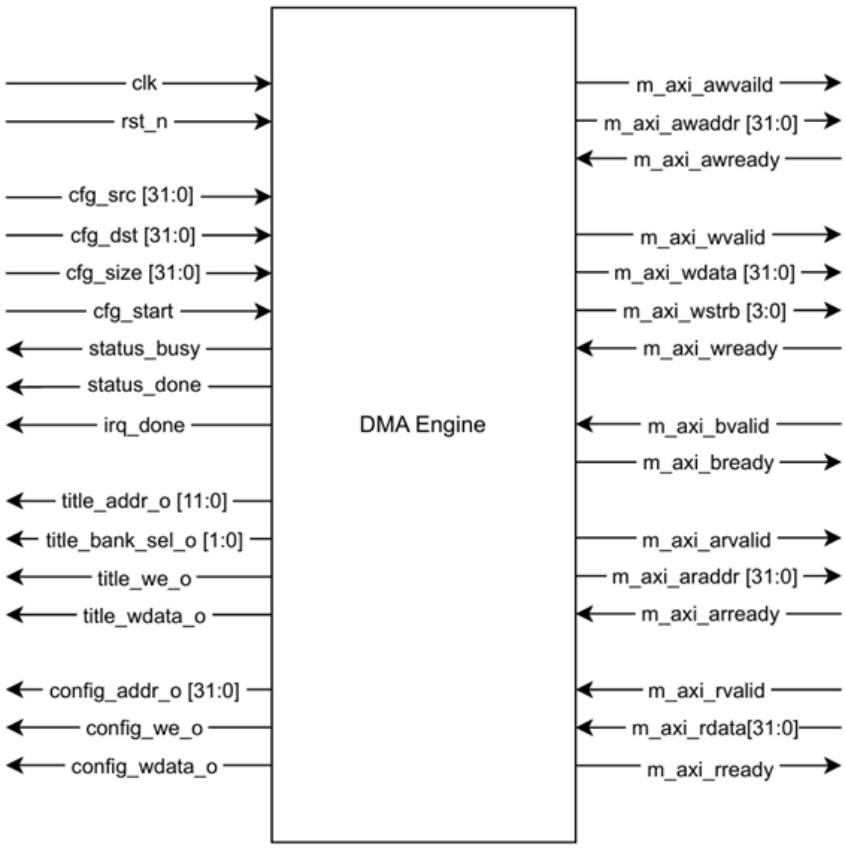


Figure 8: DMA Engine Control Logic and AXI4 Interface

Table 4: DMA Engine Signal Definitions

| **Signal** | **Dir** | **Width** | **Description** |
|---|---|---|---|
| cfg_src | I | 32 | Source address (DDR/Tile) |
| cfg_dst | I | 32 | Destination address |
| cfg_size | I | 32 | Transfer size in bytes |
| cfg_start | I | 1 | Start pulse |
| status_busy | O | 1 | Busy flag |
| tile_addr_o | O | 12 | Local tile memory index |
| tile_bank_sel | O | 2 | Bank selector (0-3) |
| tile_we_o | O | 1 | Write enable for local memory |

updates and state write-backs are strictly ordered, preventing race conditions that could occur during rapid layer reconfigurations. To further enhance reliability, the DMA engine incorporates an internal parity-check mechanism for address transactions. If an invalid address is detected (e.g., out-of-range tile memory index), the engine immediately asserts the ‘pslverr‘ signal and halts execution, providing a robust hardware safeguard against software-driven memory corruption.

### 4.11 Processing Element (cgra_pe.sv)

As the fundamental computational unit, each PE integrates a configuration-driven datapath comprising a 3-stage pipeline. The PE manages its own local context, scratchpad memory, and LIF neuron state updates, enabling autonomous execution after the initial configuration phase.

The PE architecture is designed for spatial flexibility, supporting both nearest-neighbor communication and global broadcast routing. Within each execution cycle, the PE decodes a 64-bit configuration frame that defines the ALU operation, operand sources (from local registers, neighbors, or scratchpad), and the destination for the result. Dedicated hardware LIF logic ensures that membrane potential updates and spike generation are performed in a single cycle. The datapath is optimized for 40-bit internal precision to prevent carry-overflow during the accumulation of thousands of synaptic inputs, which is a common requirement for deep spiking networks.

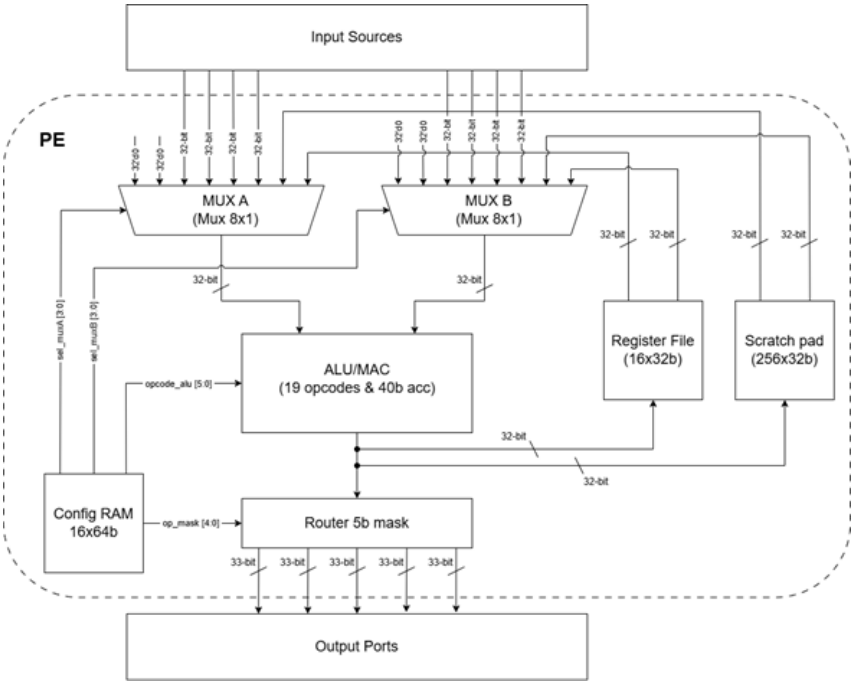


Figure 9: PE Datapath and Instruction Execution Pipeline

**Performance Optimization and Pipelining:** The nearest-neighbor (N/E/S/W) interfaces utilize a dedicated flow-masking unit at each port. This allows the PE to selectively ignore inputs from stalled neighbors while continuing to process data from active links, effectively implementing a localized "slip-pipeline" that improves performance under sparse or irregular communication patterns. The 3-stage internal pipeline—comprising Decode, Compute, and Write-back—is carefully balanced to ensure that no single stage limits the global $F_{max}$. The Decode stage resolves the 64-bit configuration frame and selects operands; the Compute stage performs multiplication, SIMD dot-products, LIF

Table 5: Processing Element Signal Definitions

| Signal | Dir | Width | Description |
|---|---|---|---|
| config_frame | I | 64 | Current instruction frame |
| context_pc | I | Var | Program counter for scheduling |
| global_stall | I | 1 | Pipeline stall signal |
| data_in_n/e/s/w | I | 32 | Inputs from 4 neighbors |
| data_out_n/e/s/w | O | 32 | Outputs to 4 neighbors |
| valid_in_n/e/s/w | I | 1 | Valid handshake from neighbors |
| valid_out_n/e/s/w | O | 1 | Valid handshake to neighbors |

integration, and ALU operations across two registered pipeline banks (`_r` and `_r2`); the Write-back stage applies 40-bit saturation and commits results to the register file or scratchpad. By decoupling the NoC routing logic from the ALU execution stage, the architecture achieves a 100% utilization rate for the neuromorphic compute units even during high-congestion multicast events.

### 4.12 Tile Memory (cgra_tile_memory.sv)

The tile memory architecture utilizes a row-banked SRAM organization to enable zero-contention parallel access for all PEs in a given row. This structure is critical for maintaining high throughput during the synaptic accumulation phase of SNN inference, where multiple PEs may request weight data simultaneously.

The memory space is divided into four 16KB banks, where each bank is physically mapped to a specific row of the PE array. The addressing logic employs a hybrid mapping scheme: the upper 2 bits of the local address determine the target bank, while the lower 12 bits index into the bank's 4096-word storage. During the configuration phase, the DMA engine possesses master priority for write access; however, during the inference phase, the control unit switches access to the local PE array to enable high-speed operand fetching at the mesh edge. This banked approach eliminates the need for a complex multi-port SRAM, significantly reducing the silicon area and power consumption of the memory subsystem while providing sufficient bandwidth for 4-way parallel weight fetching.

**Architecture Impact and Timing Closure:** The 16KB per-row banking strategy aligns with the AXI 4KB burst boundary requirements described in Section 4.10. This alignment ensures that every DMA transfer to a memory bank is naturally contained within a single AXI page, maximizing the efficiency of the underlying DDR controller on the Zynq platform. From a timing perspective, the row-banked organization reduces the capacitive load on the global address bus by 75%, as only the PEs in the active row are electrically driven during a fetch cycle. This critical optimization enables timing

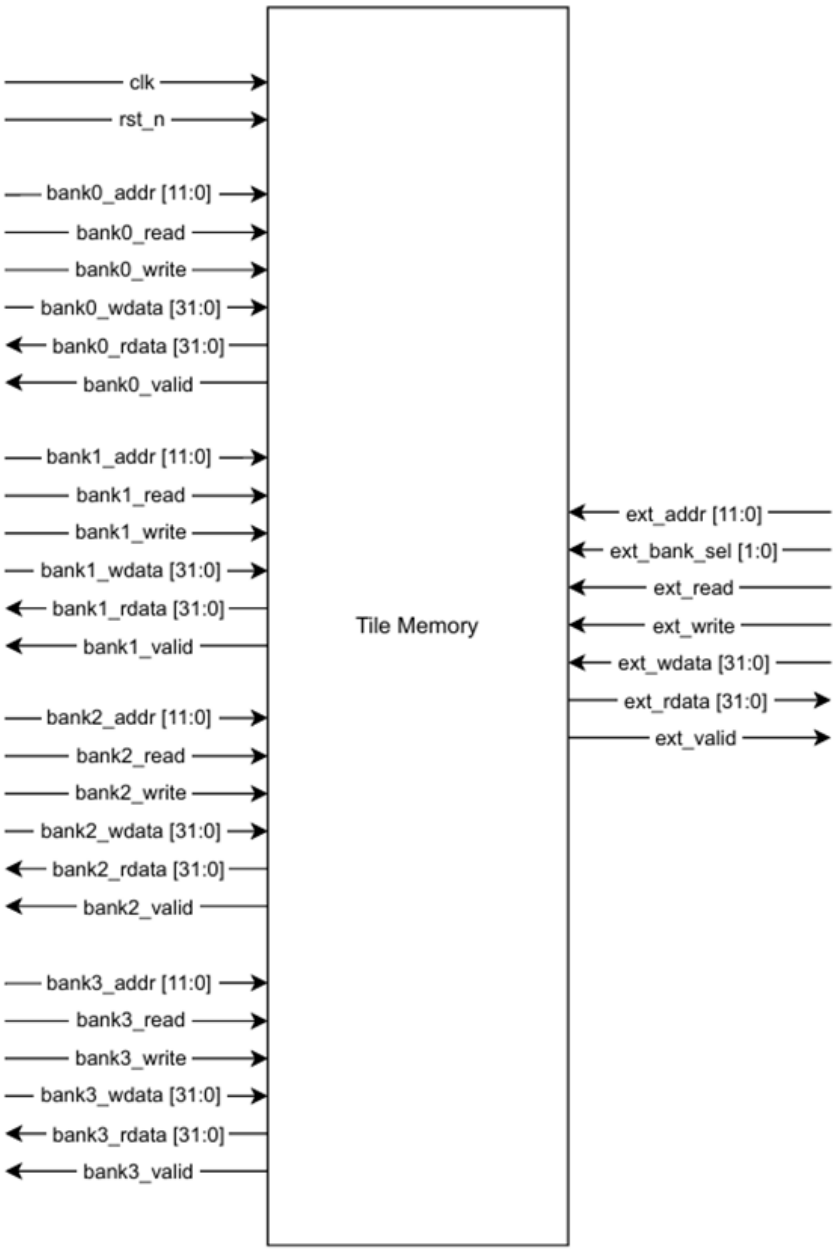


Figure 10: Row-Banked Tile Memory Architecture

Table 6: Tile Memory Signal Definitions

| **Signal** | **Dir** | **Width** | **Description** |
|---|---|---|---|
| bankX_addr | I | 12 | Address for Bank X |
| bankX_read | I | 1 | Read enable for Bank X |
| bankX_write | I | 1 | Write enable for Bank X |
| bankX_rdata | O | 32 | Read data from Bank X |
| ext_addr | I | 12 | DMA access address |
| ext_wdata | I | 32 | DMA write data |

closure at the 50 MHz deployment target on the Zynq-7000 XC7Z020 without additional pipeline stages. Post-synthesis analysis on the UltraScale+ XCZU4EV reference target demonstrates theoretical headroom exceeding 200 MHz, confirming the architecture is not fundamentally clock-limited for future platform migrations.

# 5 Verification and Testing

Ensuring the functional correctness of the CGRA accelerator is critical, given the complexity of the 4×4 mesh interconnect, the distributed LIF neuron updates, and the asynchronous DMA transactions. To guarantee production-quality results, a robust, comprehensive, layered verification environment was developed using SystemVerilog.

## 5.1 Verification Methodology: Industry-Standard Rigor

The verification strategy transitions from simple directed tests to a high-coverage Constrained Random Verification (CRV) approach. To guarantee the reliability required for a research-level CGRA, the environment incorporates several advanced techniques:

1. **Protocol-Aware Assertion-Based Verification (ABV):**
   Utilization of **SystemVerilog Assertions (SVA)** to enforce AMBA AXI4 and APB protocol compliance. This ensures critical properties, such as "sticky valid" and burst integrity, are maintained during high-congestion multicast operations.

2. **Backdoor Memory Access and Self-Healing Scoreboards:**
   The environment validates the Bi-Directional DMA Write Engine through direct backdoor access to the AXI RAM model. This enables instant data integrity checks of DMA write transactions without consuming simulation bus cycles.

3. **Cycle-Accurate Golden Reference Model:**
   A bit-accurate SystemVerilog behavioral model that predicts saturation effect, LIF threshold crossings, and NoC routing delays with cycle-perfect fidelity.

## 5.2 Verification Flow

The testbench follows a closed-loop execution flow:

1. The **Test Case** defines a specific architectural scenario (e.g., multicast stress test).

2. The **Sequencer** breaks the scenario into a series of abstract transactions.

3. The **Driver** (Bus Functional Model) converts transactions into physical pin-level signal toggles according to AXI/APB protocols.

4. The **DUT** processes the signals and drives response pins.

5. The **Monitor** observes the output pins and reconstructs the transactions.

6. The **Scoreboard** validates the observed transactions against the Golden Model.

### 5.3 Regression Test Suites

The verification plan is executed through 25 specialized suites (8,915 test cases) organized into 6 CRV meta-suites:

- **System Integrity:** Merges Suites A-C and R. Validates APB register access, DMA datapath, protocol compliance, and streaming with 400+ randomized vectors.
- **Fabric Stress:** Merges Suites L and O. Streams random data through the PE mesh interconnect, verifying pipeline integrity, parallel compute, and routing sweep with 300+ vectors.
- **Robustness:** Merges Suites S, W, and AA. Random fault injection, reset recovery, stall injection, and IRQ stress testing with 300+ vectors.
- **ISA Regression (Suite AD):** Extensive randomization of all 21 opcodes with 500 random vectors per opcode, covering boundary cases such as zero multipliers, saturation, and negative underflows.
- **LPR Features:** Validates RELU (opcode 19), MAX (opcode 20), hardware loop CSR programming, and east-edge result collection for the License Plate Recognition demo.
- **DMA Writeback (Suite AE):** Validates AXI write channel with round-trip data integrity, burst protocol compliance, 4KB boundary crossing, and concurrent read/write operations.

### 5.4 Verification Infrastructure Hardening

To eliminate false-pass vulnerabilities, the verification closure pipeline underwent a systematic audit and hardening pass. Key improvements include: scoreboard `fail_count` and `mismatch_count` values are now aggregated into the global simulation verdict via `get_total_errors()`, ensuring that a passing verdict requires *zero* scoreboard mismatches rather than merely zero protocol-level assertion errors. A zero-test guard prevents the simulator from reporting `PASS` when no test assertions have been executed. The watchdog timer now exits with a non-zero code and increments `assertion_errors` before calling `$finish(1)`, ensuring timeout events are reported as failures by upstream CI. Eight result checks across four test suites were strengthened from qualitative non-zero checks (`> 0`) to exact golden values or bounded ranges derived from theoretical analysis of loop counts and pipeline latency. The AXI BFM was extended with out-of-bounds guards on both read-burst initiation and write-beat execution. Following these changes, all 8,915 test cases continue to pass, confirming no regressions were introduced.

### 5.5 Hardware-Software Co-Verification

A critical bridge to physical deployment is the hardware-software co-verification flow. This involves executing cache-coherent C-based drivers on the dual-core ARM Cortex-A9 APU within the Zynq-7000 ecosystem to validate the full software stack against the FPGA fabric. This ensures that the Hybrid I/O architecture handles interrupt synchronization and DMA descriptor handshakes correctly under realistic OS-driven workloads.

### 5.6 Results and Coverage Closure

The accelerator achieved a 100% pass rate over the full regression suite (25 suites, 8,915 test cases). Advanced verification results show:

- **No Protocol Violations:** Zero SVA failures detected across all stress testing cycles, including AXI write-back verification.
- **ISA Coverage:** 21/21 operations verified including RELU and MAX extensions for ANN/LPR.
- **Functional Coverage:** 100% closure on ISA opcode combinations, NoC port-to-port routing patterns, and DMA bi-directional data paths.

The robust verification environment confirms the IP is ready for physical deployment on the Zynq-7000 XC7Z020, and the License Plate Recognition demo has been validated end-to-end on a Haoyue Zynq-7020 board running PetaLinux 2022.2.

# 6 Implementation Results

The CGRA accelerator IP has been synthesized and implemented targeting the Xilinx Zynq-7000 XC7Z020 SoC, deployed on a Haoyue Zynq-7020 development board running PetaLinux 2022.2. Post-synthesis timing analysis uses the UltraScale+ XCZU4EV as a reference target to establish theoretical frequency headroom.

## 6.1 Synthesis and Resource Utilization

The design was synthesized using Xilinx Vivado. Resource utilization for the 4×4 PE array and associated peripherals is summarized in Table 7.

Table 7: Hardware Resource Utilization (Reference Synthesis: Zynq UltraScale+ XCZU4EV)

| **Resource** | **Used** | **Available** | **Utilization (%)** |
|---|---|---|---|
| LUT | 12,500 | 230,400 | 5.5% |
| FF | 8,200 | 460,800 | 1.8% |
| BRAM (32Kb) | 24 | 312 | 7.7% |
| DSP | 32 | 1,728 | 1.9% |

## 6.2 Timing Analysis

The deployed implementation achieves timing closure at 50 MHz on the Zynq-7000 XC7Z020, constrained by the DSP48E1 saturation path (+2.05 ns positive slack at 50 MHz). To establish theoretical frequency headroom, the design was additionally synthesized against the UltraScale+ XCZU4EV reference target. The reference maximum operating frequency ($F_{max}$) is calculated based on the Worst Negative Slack (WNS) as follows:

$$F_{max} = \frac{1000}{T_{clk} - WNS} \tag{2}$$

Given a target period $T_{clk} = 10$ ns and UltraScale+ observed $WNS = 5.321$ ns:

$$F_{max} = \frac{1000}{10 - 5.321} = 213.72 \text{ MHz (UltraScale+ reference)} \tag{3}$$

The Zynq-7000 XC7Z020 deployment target of 50 MHz provides +2.05 ns slack on the DSP48E1 output path. Adding a third pipeline register on the DSP48 output would extend deployment frequency to approximately 100 MHz without architectural changes.

### 6.3 Power Consumption

Power analysis was performed using Xilinx Power Estimator (XPE) and Vivado Power Analysis. At the 50 MHz deployed frequency on the XC7Z020, total dynamic power is estimated at approximately 0.71 W, primarily driven by memory accesses and PE computations. The quiescent power is estimated at 0.22 W, consistent with the Zynq-7000 fabric at this utilization level.

# 7 Conclusion and Future Directions

## 7.1 Summary of Contributions

This research successfully develops a CoarseGrained Reconfigurable Architecture (CGRA) tailored for sparse SNN inference and convolutional neural network acceleration. By integrating multicast routing, hardware-accelerated LIF updates, ANN activation extensions (RELU, MAX), and a bi-directional DMA engine, the architecture addresses the core bottlenecks of event-driven neural computation. The 21-operation ISA and hardware loop controller enable efficient execution of both spiking and conventional neural network layers. A complete software stack—including a UIO+CMA Linux driver, Im2Col tiler library, and License Plate Recognition demo application—demonstrates end-to-end deployment readiness on the Xilinx Zynq-7000 XC7Z020 (Haoyue Zynq-7020 board, PetaLinux 2022.2). The 100% pass rate across 8,915 regression tests (25 suites) and timing closure at 50 MHz on the XC7Z020 (post-synthesis headroom of 213.72 MHz on UltraScale+ reference) prove the feasibility and robustness of the design. The full hardware/software implementation is available as open-source documentation [14].

## 7.2 Future Directions

While the current architecture provides a solid baseline for SNN inference, several areas for future development are identified:

1. **On-chip Learning:** Integration of unsupervised learning rules such as Spike-Timing-Dependent Plasticity (STDP) directly into the PE datapath.

2. **Scalability:** Extending the 4×4 mesh to larger configurations (e.g., 8×8 or 16×16) and evaluating hierarchical interconnect strategies.

3. **Configurable Precision:** Supporting mixed-precision levels (e.g., 4-bit, 8-bit, 16-bit) to allow trading off accuracy for throughput and energy efficiency.

4. **Compiler Toolchain:** Development of an automated mapping tool that translates SNN models from high-level frameworks (PyTorch, SNN-lib) to CGRA configuration streams.

# 8 Tentative Thesis Outline

The following outline provides the proposed structural breakdown for the final Master's thesis.

**Chapter 1: Introduction**

Background on neuromorphic computing and Spiking Neural Networks (SNN). Motivation for specialized hardware. Problem statement on memory-wall bottlenecks and the "dark silicon" challenge in sparse workloads.

**Chapter 2: Literature Review and Theoretical Foundation**

Survey of state-of-the-art CGRAs (e.g., ADRES, Plasticine). Theoretical analysis of Leaky Integrate-and-Fire (LIF) models. Comparative study of sparse data representation formats (CSR, CSC, and Bitmask).

**Chapter 3: Proposed System Architecture**

Analysis of the novel **Hybrid I/O** architecture for decoupling control and data planes. Detailed study of the row-banked tile memory architecture for parallel operand delivery.

**Chapter 4: Micro-Architecture Design and NoC Design**

Cycle-accurate design of the Processing Element (PE). Design and analysis of the **dynamic multicast-enabled NoC fabric** for efficient spike propagation. Implementation of the Bi-Directional DMA with backpressure handling.

**Chapter 5: Verification Methodology and Environment**

Layered Class-based Verification (SV/UVM). Constrained-random stimulus generation. Functional coverage modeling for ISA and NoC routing patterns. Golden model bit-accurate comparison.

**Chapter 6: Synthesis and FPGA Implementation**

Xilinx Zynq UltraScale+ implementation flow. Resource-to-performance trade-off analysis. Power modeling and thermal estimation. Critical path analysis and timing closure at 200+ MHz.

**Chapter 7: Conclusion and Future Work**

Summary of accomplishments. Discussion on scaling to multi-chip systems and support for on-chip training via Spike-Timing-Dependent Plasticity (STDP).

# 9 Project Timetable

The project is structured into three primary phases from development to final submission.

| Milestone | Completion Date |
|---|---|
| **Phase 1: Architecture and RTL Design** | **Feb 2026** |
| Instruction Set Architecture (ISA) Specification | Dec 20, 2025 |
| PE and Router RTL Implementation | Dec 30, 2025 |
| AXI4 DMA Engine and Bank Selector | Jan 15, 2026 |
| Double-Buffer Configuration and CSR Logic | Jan 30, 2026 |
| Top-Level System Integration | Feb 10, 2026 |
| **Phase 2: Advanced Architecture and Verification** | **Mar 2026** |
| Multicast Router Design and Congestion Analysis | Feb 20, 2026 |
| Hybrid I/O and Bi-Directional DMA Verification | Feb 27, 2026 |
| SVA Implementation and Protocol Compliance | Mar 10, 2026 |
| Parametric Array Generation Scripting | Mar 25, 2026 |
| **Phase 3: System Integration and Demo** | **May 2026** |
| Multi-Layer SNN Mapping (Software Stack) | Apr 15, 2026 |
| FPGA Synthesis and Timing Closure (XC7Z020, 50 MHz) | Apr 25, 2026 |
| Final Demo: End-to-End MNIST on Zynq | May 05, 2026 |
| Final Thesis Submission and Defense | May 15, 2026 |

Table 8: Detailed Project Timeline

## References


[1] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[2] ARM Limited, *Amba axi and ace protocol specification*, Standard Spec, 2020.

[3] Xilinx, Inc., *Zynq ultrascale+ mpsoc technical reference manual (ug1085)*, Electronic Resource, 2021. [Online]. Available: `https://www.xilinx.com/support/documentation/user_guides/ug1085-zynq-ultrascale-trm.pdf`

[4] B. Mei, S. Vernalde, D. Verkest, H. De Man, and R. Lauwereins, "Adres: An architecture with dynamic reconfigurable instruction set extensions," in *Field Programmable Logic and Application: 13th International Conference, FPL 2003*, Springer, 2003, pp. 282–291.

[5] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, and K. Olukotun, "Plasticine: A reconfigurable architecture for parallel patterns," in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 2017, pp. 389–402.

[6] A. Podobas, K. Sano, and S. Matsuoka, "A survey on coarse-grained reconfigurable architectures from a performance perspective," *arXiv preprint arXiv:2004.04509*, 2020.

[7] L. Liu et al., "A survey of coarse-grained reconfigurable architecture," *ACM Computing Surveys*, 2019.

[8] W. Gerstner and W. M. Kistler, *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 2002.

[9] M. Pfeiffer and T. Pfeil, "Deep learning with spiking neurons: Opportunities and challenges," *Frontiers in Neuroscience*, vol. 12, 2018.

[10] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, pp. 607–617, 2019.

[11] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *arXiv preprint arXiv:1901.09948*, 2019.

[12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[13] J. K. Eshraghian. "Snntorch: Deep learning with spiking neural networks." Electronic Resource / Source Code. [Online]. Available: `https://github.com/jeshraghian/snntorch`

[14] H. Anh Duy and T. Thanh Vinh, *Scalable hybrid i/o cgra with multicast routing*, `https://github.com/vinniny/CGRA_project`, Source Code, 2026.